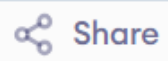


main.c



Run

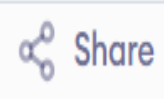
Output

```
1 #include <stdio.h>
2 int linearSearch(int arr[], int n, int key) {
3     for(int i = 0; i < n; i++) {
4         if(arr[i] == key)
5             return i;
6     }
7     return -1;
8 }
9 int main() {
10     int arr[] = {3, 5, 7, 9};
11     int n = sizeof(arr) / sizeof(arr[0]);
12     int key = 7;
13
14     int index = linearSearch(arr, n, key);
15
16     if(index != -1)
17         printf("Found at index: %d\n", index);
18     else
19         printf("Not found\n");
20     return 0;
21 }
```

Found at index: 2

=== Code Execution Successful ===

main.c



Run

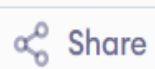
Output

```
1 #include <stdio.h>
2
3 int main() {
4     int arr[5] = {10, 20, 30, 40, 50};
5     for (int i = 0; i < 5; i++)
6         printf("%d ", arr[i]);
7     return 0;
8 }
```

10 20 30 40 50

=== Code Execution Successful ===

main.c



Share

Run

Output

```
1  #include <stdio.h>
2  int binarySearch(int arr[], int n, int key){
3      int low = 0, high = n - 1;
4      while (low <= high) {
5          int mid = (low + high) / 2;
6          if (arr[mid] == key)
7              return mid;
8          else if (arr[mid] < key)
9              low = mid + 1;
10         else
11             high = mid - 1; }
12     return -1; }
13 int main(){
14     int arr[] = {1, 3, 5, 7, 9};
15     int n = sizeof(arr) / sizeof(arr[0]);
16     int key = 7;
17     int index = binarySearch(arr, n, key);
18     if (index != -1)
19         printf("Found at index %d\n", index);
20     else
21         printf("Not found\n");
22     return 0;
```

Found at index 3

=== Code Execution Successful ===

main.c



Share

Run

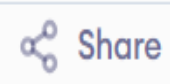
Output

```
1  #include <stdio.h>
2  #define MAX 5
3
4  int stack[MAX];
5  int top = -1;
6
7  void push(int data) {
8      if (top == MAX - 1) {
9          printf("Stack Overflow");
10         return; // stop here
11     } else {
12         stack[++top] = data; // clearer increment order
13     }
14 }
15
16 int main() {
17     push(10);
18     push(20);
19     printf("%d", stack[top]); // prints 20
20     return 0;
21 }
```

20

=== Code Execution Successful ===

main.c



Share

Run

Output

```
1  #include <stdio.h>
2  #define MAX 5
3  int stack[MAX];
4  int top = 2;
5  int pop() {
6      if (top == -1)
7          return -1;
8      return stack[top--];
9  }
10 int main() {
11     // example values
12     stack[0] = 10;
13     stack[1] = 20;
14     stack[2] = 30;
15     int x = pop();
16     printf("%d\n", x);
17     return 0;
18 }
```

30

=== Code Execution Successful ===

main.c



Share

Run

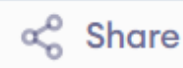
Output

```
1  #include <stdio.h>
2  #define MAX 100
3  int stack[MAX];
4  int top = 0; // if top==0 means stack is empty
5  int pop() {
6      if (top == 0) {
7          printf("Underflow\n");
8          return -1;
9      }
10     int val = stack[top - 1];
11     top--;
12     return val;
13 }
14 int main() {
15     stack[top++] = 10;
16     stack[top++] = 20;
17     printf("%d\n", pop());
18     printf("%d\n", pop());
19     printf("%d\n", pop());
20
21     return 0;
22 }
```

```
20
10
Underflow
-1
```

```
=== Code Execution Successful ===
```

main.c



Run

Output

```
1  #include <stdio.h>
2  #define MAX 5
3  void enqueue(int queue[], int *rear, int x) {
4      if (*rear == MAX) {
5          printf("Overflow\n");
6          return;
7      } else {
8          queue[*rear] = x;
9          (*rear)++;
10         printf("Value is %d\n", x);
11     }
12 }
13 int main() {
14     int queue[MAX], x;
15     int front = 0, rear = 0; // front is unused in enqueue-only code
16     printf("Enter the value: ");
17     scanf("%d", &x);
18     enqueue(queue, &rear, x);
19     return 0;
20 }
```

Enter the value: 12

Value is 12

=== Code Execution Successful ===

main.c



Share

Run

Output

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node *next;
6  };
7  int main() {
8      struct Node *newNode;
9
10     newNode = (struct Node *)malloc(sizeof(struct Node));
11     if (newNode == NULL) {
12         printf("Memory allocation failed\n");
13         return 1;
14     }
15     newNode->data = 10;
16     newNode->next = NULL;
17     printf("%d", newNode->data);
18     free(newNode);
19     return 0;
20 }
```

10

=== Code Execution Successful ===

main.c



Share

Run

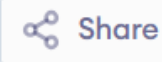
Output

```
1  #include <stdio.h>
2  #include <stddef.h>
3  struct Node {
4      int data;
5      struct Node *next;
6  };
7  void display(struct Node *head)
8  {
9      while (head != NULL)
10     {
11         printf("%d ", head->data);
12         head = head->next;
13     }
14 }
15 int main(void)
16 {
17     struct Node a = {1, NULL};
18     display(&a);
19     return 0;
20 }
```

1

=== Code Execution Successful ===

main.c



Run

Output

```
1  #include <stdio.h>
2  #define SIZE 5
3  int queue[SIZE];
4  int front = -1, rear = -1;
5  void enqueue(int item){
6      if (front != -1 && ((rear + 1) % SIZE == front)){
7          printf("Queue Full\n");
8          return;}
9      if (front == -1)
10         front = 0;
11     rear = (rear + 1) % SIZE;
12     queue[rear] = item;}
13 void display(){
14     if (front == -1){
15         printf("Queue Empty\n");
16         return;}
17     printf("Queue elements: ");
18     int i = front;
19     while (1) {
20         printf("%d ", queue[i]);
21         if (i == rear)
22             break;
```

Queue elements: 10 20 30

=== Code Execution Successful ===